

CSE 511: Hot Spot Analysis Project Report

Name: Sejal Patil | ASU ID: 1239407081 | email id: sapat11@asu.edu

Reflection:

For this project, I performed spatial hot spot analysis on a NYC taxi trip dataset using Apache Spark 3.4.1 and Scala 2.12.17. The development environment was configured on Windows 10 OS with Java JDK 17 and sbt 1.10.10.

I completed two different hot spot analysis tasks. First was Hot Zone Analysis in which I modified the HotzoneUtils.scala and HotzoneAnalysis.scala files, and second was Hot Cell Analysis in which I modified the HotcellUtils.scala and HotcellAnalysis.scala files. Understanding Entrance.scala was vital as it initiated the Spark session, parsed command-line arguments, and invoked functions based on specified queries.

Lessons Learned:

This project was my first real experience working with spatial data at scale. It deepened my understanding of spatial statistical concepts, particularly the application of the Getis-Ord statistic in identifying hot spots. I also learned spatial relationships and operations which are critical for problems involving location data like range joins. This project also taught me patience and systematic debugging. Each test run took several minutes, so I couldn't just try random fixes. I had to think carefully, test incrementally, and plan my approach. The setup process itself was a learning experience—installing all the dependencies, configuring environment variables, and getting everything to work together on Windows took longer than I anticipated. While running the project I faced Scala version compatibility issues but going through that process gave me a better understanding of how these big data tools fit together. Overall, this project bridged the gap between theoretical and practical implementation, showing me how spatial analytics can solve real problems.

Implementation:

Hot Zone Analysis:

Hot Zone Analysis helps us to identify which parts of the city see the most taxi activity by counting how many pickups happen within predefined rectangular zones. The main function ST_Contains determines whether a specific pickup point falls inside a specific rectangle. The function gets two strings: one with four coordinates for the rectangle's corners (x1, y1, x2, y2) and one with two coordinates for the point (x, y). The function first figures out the actual boundaries by finding the minimum and maximum of x and y values. Then it just checks if the point's coordinates fall within those boundaries. If the point's x is

between the left and right edges, and the point's y is between the bottom and top edges, then the point is inside the rectangle and the function returns true.

```
def ST_Contains(queryRectangle: String, pointString: String): Boolean = {
  val rectArray = queryRectangle.split(",")
  val x1 = rectArray(0).trim.toDouble
  val y1 = rectArray(1).trim.toDouble
  val x2 = rectArray(2).trim.toDouble
  val y2 = rectArray(3).trim.toDouble
  val xMin = Math.min(x1, x2)
  val xMax = Math.max(x1, x2)
  val yMin = Math.min(y1, y2)
  val yMax = Math.max(y1, y2)
  val pointArray = pointString.split(",")
  val x = pointArray(0).trim.toDouble
  val y = pointArray(1).trim.toDouble
  (x >= xMin && x <= xMax && y >= yMin && y <= yMax)
}
```

The filtered results are then aggregated by grouping on rectangle identifiers and counting the points in each zone, with results sorted alphabetically by rectangle name. The persist() keeps the data in memory for efficiency and coalesce(1) combines all Spark partitions into a single csv file instead of multiple files.

```
val orderedJoinDf = spark.sql("select rectangle, count(point) from joinResult group by rectangle order by rectangle").persist()
orderedJoinDf.createOrReplaceTempView("orderedJoin")
return orderedJoinDf.coalesce(1) // YOU NEED TO CHANGE THIS PART
```

Hotcell Analysis:

Hot Cell Analysis identifies statistically significant hot spots in NYC taxi pickup data using the Getis-Ord statistic. For the hot cell analysis task, yellow_trip_sample_100000.csv dataset was used as input. I began by filtering the data to keep only cells within the study boundaries and then aggregated pickup points by their (x, y, z) coordinates to count pickups per cell. In HotcellUtils.scala, I wrote the countNeighbors function that loops through all 27 possible neighbor positions and determines how many valid neighbors each cell has. I then used a self-join query with BETWEEN conditions to find all neighbors within each specified dimension and sum their pickup counts. Before calculating z-scores, I calculated total count, mean, variance, and standard deviation. Finally, I implemented the calculateGScore function that applies the Getis-Ord formula, which compares observed neighborhood activity against expected values adjusted for spatial autocorrelation. The final query applies this function to all cells and sorts in descending order based on their scores. The top 50 cells with the highest G*-scores are selected as the "hottest" cells. These coordinates are written to output file without the actual z-score values. As in the previous analysis, I had to add ,coalesce(1) to generate 1 csv file. This concludes the hot spot analysis tasks on the given datasets using Apache Spark. The coded solutions were packaged into a JAR file and executed on a Spark cluster to perform distributed computation and obtain results.

Conclusion:

This concludes the hot spot analysis tasks on the given datasets using Apache Spark. The coded solutions were packaged into a JAR file and executed on a Spark cluster using spark-submit to perform distributed computation and obtain results. Here, left side is the hot cell analysis csv output file and right side is the hot zone analysis csv output file.

part-00000-070814e3-31c6-4aef-bdbb-8924a5fc0e78-c000.csv	part-00000-7c3eda8f-af04-47f1-8e12-ff08bf145ed8-c000.csv
1 -7399,4075,15	1 "-73.789411,40.666459,-73.756364,40.680494",1
2 -7399,4075,22	2 "-73.793638,40.710719,-73.752336,40.730202",1
3 -7399,4075,14	3 "-73.795658,40.743334,-73.753772,40.779114",1
4 -7399,4075,29	4 "-73.796512,40.722355,-73.756699,40.745784",1
5 -7398,4075,15	5 "-73.797297,40.738291,-73.775740,40.770411",1
6 -7399,4075,16	6 "-73.802033,40.652546,-73.738566,40.668036",8
7 -7399,4075,21	7 "-73.805770,40.666526,-73.772204,40.690003",3
8 -7399,4075,28	8 "-73.815233,40.715862,-73.790295,40.738951",2
9 -7399,4075,23	9 "-73.816380,40.690882,-73.768447,40.715693",1
10 -7399,4075,30	10 "-73.819131,40.582343,-73.761289,40.609861",1
11 -7398,4075,14	11 "-73.825921,40.702281,-73.790734,40.719217",2
12 -7399,4074,15	12 "-73.826577,40.757744,-73.790317,40.779587",1
13 -7398,4075,22	13 "-73.832707,40.620010,-73.746541,40.665414",200
14 -7399,4075,27	14 "-73.839460,40.746988,-73.815375,40.781725",3
15 -7398,4075,29	15 "-73.840130,40.662481,-73.801134,40.691956",4
16 -7398,4075,23	16 "-73.840817,40.775618,-73.790584,40.801020",1
17 -7399,4074,28	17 "-73.842332,40.804005,-73.790064,40.845347",2
18 -7399,4074,22	18 "-73.843148,40.701398,-73.816380,40.715998",2
19 -7399,4074,16	19 "-73.849479,40.681155,-73.810634,40.704221",2
20 -7398,4076,15	20 "-73.861099,40.714345,-73.817260,40.764083",21
21 -7398,4075,21	21 "-73.862040,40.706406,-73.829798,40.739016",16
22 -7399,4075,13	22 "-73.864482,40.833000,-73.851686,40.842478",1
23 -7399,4075,9	23 "-73.867911,40.734291,-73.847510,40.748860",1
24 -7398,4075,23	24 "-73.869236,40.765468,-73.825139,40.798517",136
25 -7398,4075,30	25 "-73.873659,40.744942,-73.853128,40.758349",1
26 -7398,4075,16	26 "-73.875093,40.711476,-73.856258,40.735947",3
27 -7399,4074,30	27 "-73.876389,40.756642,-73.854497,40.771776",67
28 -7398,4076,14	28 "-73.878249,40.581702,-73.766710,40.639962",10
29 -7398,4076,28	29 "-73.884652,40.822974,-73.856153,40.833622",1
30 -7399,4074,29	30 "-73.891752,40.727801,-73.864472,40.749165",8
31 -7398,4075,27	31 "-73.891834,40.763673,-73.855049,40.786046",334
32 -7399,4074,14	32 "-73.897174,40.822518,-73.872773,40.840171",2
33 -7398,4076,29	33 "-73.899862,40.746820,-73.869426,40.771545",12
34 -7399,4074,21	34 "-73.901227,40.834354,-73.886599,40.843533",1
35 -7399,4075,8	35 "-73.903077,40.800183,-73.868114,40.827664",1
36 -7398,4075,13	36 "-73.903571,40.759641,-73.893497,40.767425",1
37 -7399,4074,28	37 "-73.904774,40.706462,-73.859631,40.730680",1
38 -7398,4076,22	38 "-73.910492,40.829112,-73.890277,40.861538",2
39 -7399,4075,17	39 "-73.910690,40.731601,-73.877084,40.748728",10
40 -7398,4075,9	40 "-73.916775,40.820484,-73.886014,40.835508",3
41 -7399,4074,27	41 "-73.919040,40.727433,-73.894665,40.765790",19
42 -7398,4076,30	42 "-73.920588,40.884633,-73.896359,40.915541",1
43 -7399,4075,7	43 "-73.922754,40.667106,-73.900048,40.687222",1
44 -7398,4076,26	44 "-73.923983,40.765866,-73.887708,40.790954",9
45 -7399,4074,17	45 "-73.924042,40.691451,-73.877613,40.717801",3
	46 "-73.924124,40.712865,-73.887079,40.735202",2
	47 "-73.924905,40.873248,-73.896617,40.890453",1
	48 "-73.926629,40.812290,-73.901294,40.824395",1
	49 "-73.927318,40.818021,-73.905017,40.843211",3
	50 "-73.928722,40.855672,-73.910431,40.873262",2